Handwritten Digit Classification

Jeremy Chernin

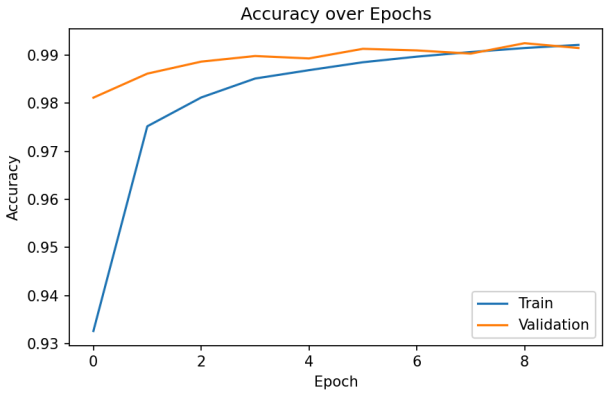
2/28/2026

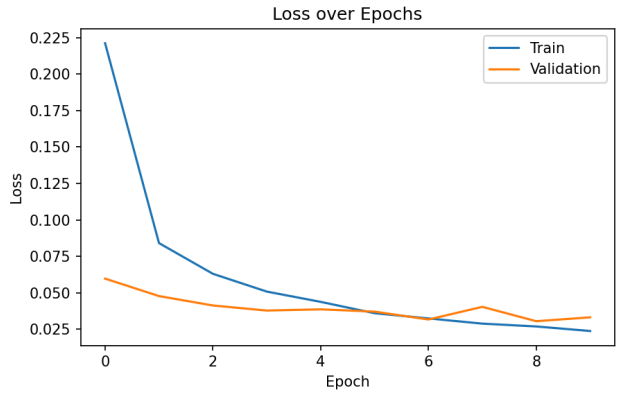
Problem Definition & Dataset Curation

For this project, I attempted to classify handwritten digits into 10 categories (digits 0 through 9) by using a convolutional neural network. This classification problem has real applications in areas where handwritten text needs to be parsed, such as reading signatures or any kind of handwritten form that is submitted online. I will be using the MNIST dataset for this model, which has over 70,000 images of handwritten digits that are preprocessed and cleaned for training use. Very minimal preprocessing was needed to prepare the images for classification, since MNIST is a common dataset that has preprocessing steps already applied. The major preprocessing step that I implemented was changing the range of the pixel color values to [0, 1] instead of the [0, 255] that the dataset uses since this is the format that PyTorch expects, and so it was required before training the model. After preprocessing, the script creates a validation group containing 10% of the training images, and then creates a training group with the other 90% of the images in order to train the model.

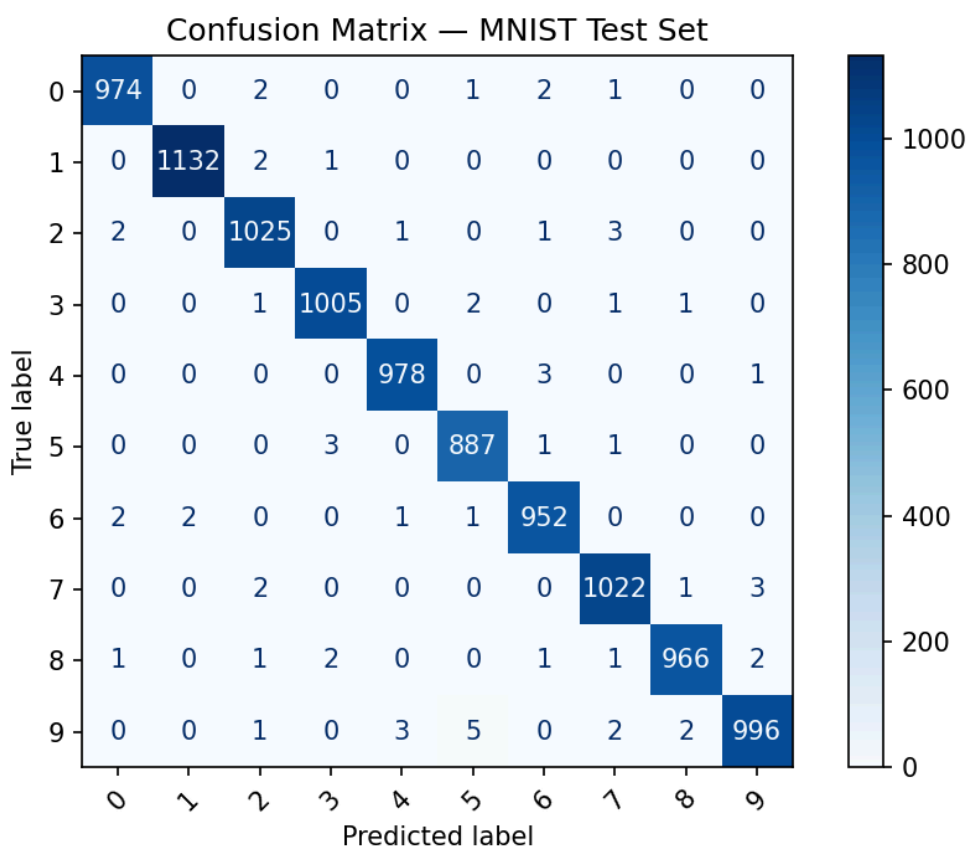
Deep Learning Models

For this project, I used a convolutional neural network which is perfect for this type of image classification problem. I asked Claude to help generate the code to train the neural network, and it gave me a script that downloaded the MNIST dataset automatically, normalized the images, and then split the dataset into training, validation, and test sets. It then set up a loop to train the model, which computed loss during each step and updated weights automatically. This model uses two convolution layers, with the first using 32 3x3 filters and the second using 64 3x3 filters. It also implements max-pooling, which it uses to reduce the number of features it processes in order to make computation easier. Finally, it ran the model against the test dataset and showed the test accuracy while visualizing the results, outputting two graphs that show a summary of the training process. The “Accuracy over Epochs” graph shows a 99% accuracy rate by the 10th epoch, proving that the model was incredibly accurate at determining which digit it saw. Below are the two charts that the script created, displaying the accuracy and loss over the course of the training time, showing how it improved with each epoch until the accuracy settled around 99% and the loss converged to around 0.03.



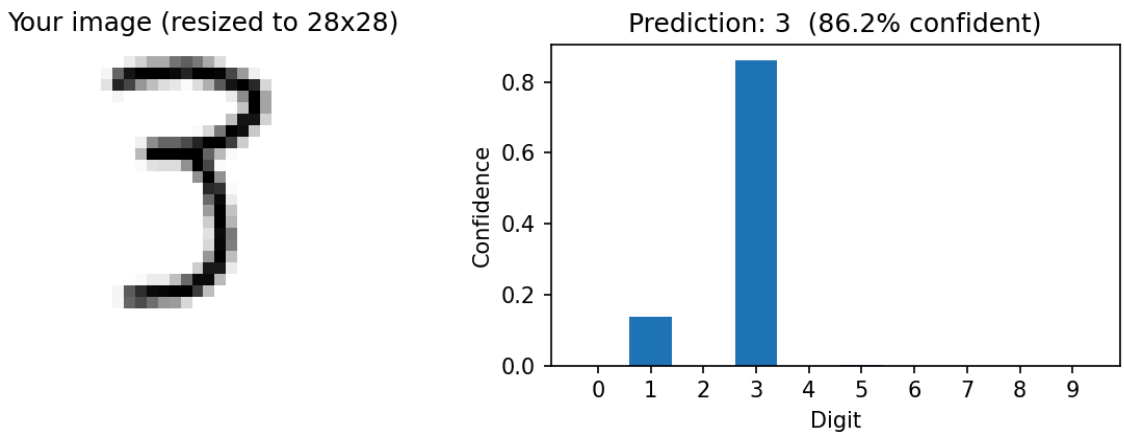


The script also generated a confusion matrix, showing where it made mistakes. For example, it seems to more commonly mistake a number the number 9 as a 5, since the number in the matrix at row 9 column 5 is “5” which is higher than most of the other entries. This makes sense, since the shapes of 9 and 5 are pretty similar and so it would be harder to distinguish between these two. However, this error is relatively insignificant compared to the diagonal of the matrix, which means that it almost always correctly predicts the digit it is given, since the diagonal represents correct predictions (where the “true label” and “predicted label” are the same).



Results

The results of this training were pretty incredible. With around 99% accuracy after 10 epochs, it can confidently predict any number thrown at it with a loss of only around 0.03. The confusion matrix re-emphasizes this fact, showing that misclassification happens very infrequently, usually only with numbers that a human could mistake as well, such as the numbers 3 and 8 where sloppy handwriting could have caused errors due to their similar shapes. In order to test the model with a real scenario, I drew the number 3 in a photo editing app and saved the image, and it was able to correctly predict that the number was a 3 with 86.2% confidence. This is especially impressive because the images in the MNIST dataset are pre-processed to work well with convolutional neural networks, with things like consistent cropping and image sizing. However, the high confidence in this example shows that this model can generalize to real world situations where the data fed into it isn't as clean as the MNIST dataset.



Lessons & Experience

Throughout the course of this project, I learned that convolutional neural networks are a powerful tool that can complete classification problems very quickly and easily. I ran into a few challenges during this project, but I was able to debug them and get a working project. For example, I originally created the script to use TensorFlow, but this doesn't support Python 3.14, which is what I was running on my system. This forced me to switch to PyTorch, which worked just as well. I also had some trouble setting up a virtual environment, since I am not very familiar with Python. I was able to find the commands I needed to get it working, and had no issues after that. By looking at the confusion matrix, I was also able to see where the model makes mistakes. This is usually in the places you would expect, such as mistaking a 3 for an 8, since the shapes of the digits are similar. The most impressive part to me was how few epochs were needed in order to create a very accurate model, with the training time taking less than a minute on my computer. This seemed pretty impressive to me, and shows the power of convolutional neural networks with this type of problem.